



南開大學
Nankai University

计算机学院
计算机网络实验报告

利用流式套接字编写聊天程序

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2025 年 11 月 15 日

目录

1 实验要求与任务	2
2 系统分析与总体设计	2
2.1 通信协议与报文格式	2
2.2 模块划分与依赖关系	3
2.3 线程与并发策略	3
2.4 关键数据结构与可靠性	3
3 逐步具体实现	4
3.1 协议与公共工具层 (common)	4
3.2 多线程服务器 (server)	5
3.3 命令行客户端 (client)	6
3.4 图形界面客户端 (client_gui)	7
3.5 构建与运行	7
4 运行与验证	7
5 实验总结与改进方向	9

1 实验要求与任务

实验要求：

1. 设计聊天协议，并给出聊天协议的完整说明。
2. 利用 C 或 C++ 语言，使用基本的 Socket 函数进行程序编写，不允许使用 CSocket 等封装后的类。
3. 程序应有基本的对话界面，但可以不是图形界面。程序应有正常的退出方式。
4. 完成的程序应能支持英文和中文聊天。
5. 采用多线程，支持多人聊天。
6. 编写的程序应结构清晰，具有较好的可读性。
7. 在实验中观察是否有数据包的丢失，提交程序源码、可执行代码和实验报告。

实验任务：

1. 使用流式 Socket，设计一个聊天协议，要求聊天信息带有时间标签。请完整地说明交互消息的类型、语法、语义、时序等具体的消息处理方式。
2. 对聊天程序进行设计。给出模块划分说明、模块的功能和模块的流程图。
3. 在 Windows 系统下，利用 C/C++ 对设计的程序进行实现。程序界面可以采用命令行方式，但需要给出使用方法。编写程序时，只能使用基本的 Socket 函数，不允许使用对 Socket 封装后的类或架构。
4. 完成的程序应能支持英文和中文聊天；采用多线程，支持多人聊天。
5. 编写的程序应结构清晰，具有较好的可读性。
6. 观察是否有数据包的丢失。

2 系统分析与总体设计

为满足自定义协议、跨平台扩展与多终端复用等要求，系统在 Windows 环境下直接调用 Winsock2 API 处理 socket 连接，同时通过 `chat_common` 静态库封装协议帧、UTF-8 设置与可靠收发逻辑，为服务器、CLI 与 GUI 应用提供统一基线。为便于维护与调试，所有公共能力集中在一处，并由不同可执行程序按需组合，从而保证通信、编码与线程策略保持一致。

2.1 通信协议与报文格式

协议采用 `kProtocolVersion=CHAT/1.0`，以 TCP 为承载信道，所有字段均按“指令 + 时间戳 + 参数 + UTF-8 Payload”组织。`SplitTokens` 与 `ParseLength` 负责把控令牌数量及长度合法性，避免出现粘包或畸形报文。

表 1: CHAT/1.0 报文类型

帧类型	说明
HELLO	HELLO <username> CLIENT/<tag>: 客户端首帧, 请求进入会话
WELCOME	WELCOME <serverName> <timestamp> CHAT/1.0: 确认服务名与协议版本
MSG	MSG <timestamp> <sender> <len>\n<payload>: 广播聊天内容
SYS	SYS <timestamp> <len>\n<payload>: 系统提示、成员进出等通知
ERR	ERR <timestamp> <len>\n<payload>: 握手失败或非法指令的错误描述
BYE	BYE <len>\n<reason>: 主动离线并附带原因

2.2 模块划分与依赖关系

系统围绕公共协议、服务器、命令行客户端、GUI 客户端四个模块展开, 在 CMake 中把 `chat_common` 编译成静态库, 其余可执行程序仅需链接 `ws2_32` 及 (GUI 场景下的) `user32/shell32` 即可运行。

表 2: 主要模块职责

模块	功能概述
common	<code>chat_protocol.h/cpp</code> 提供 <code>SendAll</code> 、 <code>ReceiveLine</code> 、 <code>NormalizeUsername</code> 等工具, 统一帧裁剪与 UTF-8 设置
server	维护 <code>std::vector<std::shared_ptr<ClientSession>></code> , 负责注册、广播、超时清理与 Ctrl+C 收尾
client	CLI 客户端, 封装参数解析、交互式输入与 <code>ReceiverLoop</code> 消息泵
client_gui	Win32 GUI, 依靠 <code>GuiState</code> 管理窗口句柄与 socket 状态, 并通过自定义 WM 消息跨线程通信

2.3 线程与并发策略

- 服务器主线程循环 `accept`, 为每个新连接创建 `std::thread(HandleClient)`, 在 `BroadcastFrame` 中通过互斥锁串行访问会话列表, 写失败即刻清理。
- CLI 客户端使用前台线程读取用户输入, 后台线程 `ReceiverLoop` 解析 MSG/SYS/ERR 帧并借助 `g_io_mutex` 与 `std::atomic<bool>` 协调状态。
- GUI 客户端采用 UI 线程 + 网络线程结构, 网络线程完成握手、收包并把日志通过 `WM_CHAT_LOG/WM_CHAT_CONNECT` 投递到 UI 线程, 避免直接跨线程操作控件。

2.4 关键数据结构与可靠性

`ClientSession` 保存 socket、用户名、远端地址及残留 buffer, 通过 `RegisterClient`、`RemoveClient` 与广播函数保持列表一致性, `ConsoleHandler` 则在 Ctrl+C 时触发 `shutdown`, 保证服务器优雅退出。GUI 侧的 `GuiState/LoginDialogState` 追踪窗口句柄、socket、后台线程与登录输入, 在 `RequestSocketShutdown`、

WM_DESTROY 等分支中回收资源，避免线程泄漏。此外所有终端都会调用 `NormalizeUsername` 与长度上限常量来过滤非法字符，配合 `SendAll/ReceiveBytes` 循环确保没有残帧或半包。

3 逐步具体实现

本节按照“公共库 → 服务器 → CLI → GUI → 构建”的顺序总结源码实现路径。

3.1 协议与公共工具层（common）

`chat_protocol.h/cpp` 负责暴露 UTF-8 控制、时间戳生成与可靠收发 API，所有终端都在入口处调用 `SetupConsoleUtf8` 以避免 Windows 控制台乱码。在收发实现上，`SendAll` 会循环使用 `send` 函数将数据分块发送到套接字直至发送完全部字节，结合 `ReceiveLine/ReceiveBytes` 的 buffer 累积机制，可有效解决粘包问题：

```
1 bool SendAll(SOCKET socket, const char* data, std::size_t length) {
2     std::size_t sent_total = 0;
3     while (sent_total < length) {
4         const int chunk = send(socket, data + sent_total,
5                               static_cast<int>(length - sent_total), 0);
6         if (chunk == SOCKET_ERROR || chunk == 0) {
7             return false;
8         }
9         sent_total += static_cast<std::size_t>(chunk);
10    }
11    return true;
12 }
```

`CurrentIsoTimestamp` 使用 `std::chrono + gmtime_s` 生成 UTC 时间戳，`NormalizeUsername` 则统一裁剪用户名，只保留字母、数字与“-“/“_“。I 满足 CLI 和 GUI 在字符处理上的一致性，common 模块额外抽象出多组工具接口。

- **SendLine**：自动补充“\n”结尾，调用端无需再拼接，然后调用 `SendAll` 发送；`SendAll` 返回失败会立即上报。
- **ReceiveLine**：利用会话 buffer 逐块拼接直到命中“\n”，再统一裁剪“\r”适配 Windows/Unix 控制台。也就是检查缓冲区中是否有完整的一行（以换行符“\n”结尾），如果有，则提取该行并返回。
- **ReceiveBytes**：从套接字接收指定字节数的数据，存储到缓冲区 buffer 中。如果缓冲区中数据量达到要求，则提取数据到 payload，并从缓冲区中移除已读取的数据。
- **SplitTokens+ParseLength**：统一拆分指令与数值字段，任何非法令牌都会输出清晰的“unknown command”。

这些接口被 server 端的 `SendFrame`、client/GUI 的 `ReceiverLoop/NetworkThreadProc` 循环复用。

3.2 多线程服务器 (server)

服务器的 `ClientSession` 结构体表示一个客户端会话, 记录 socket、用户名、远端地址、缓冲区以及标记客户端是否成功加入聊天的 bool 值, 配合全局 `std::vector<std::shared_ptr<ClientSession>>` 和互斥锁维护在线列表。典型流程如下:

1. **握手:** `HandleClient` 首先调用 `ReceiveLine` 读取 HELLO 帧, 若令牌或用户名不合法则通过 `SendError` 立即关闭连接。
2. **注册与广播:** 成功注册后发送 WELCOME, 同时调用 `BroadcastSystemMessage` 通告 “xxx joined the chat”。
3. **指令循环:** 支持 MSG 和 BYE 两类帧, 前者校验长度并使用 `BroadcastChatMessage` 分发, 后者根据附带原因构造离线提示。
4. **收尾:** 任意异常或主动退出都会进入统一的清理逻辑, 调用 `RemoveClient`、`CloseSocket` 并再次广播状态; `ConsoleHandler` 在 Ctrl+C 时置位 `g_running` 并关闭监听 socket。

为了实现该流程, 设计了多个函数来逐步实现服务端的功能要求:

- **CloseSocket(SOCKET socket):** 关闭指定的套接字。具体看是调用 `shutdown` 终止套接字的读写操作和调用 `closesocket` 释放套接字资源。
- **SendFrame(SOCKET socket, const std::string& header, const std::string& payload):** 发送一个数据帧, 包括头部和可选的负载。具体看是调用 `chat::SendLine` 发送头部; 如果负载不为空, 调用 `chat::SendAll` 发送负载。
- **BroadcastFrame(const std::string& header, const std::string& payload):** 向所有已连接的客户端广播一个数据帧。通过加锁保护全局客户端列表, 遍历每个客户端并调用 `SendFrame` 发送数据帧。如果发送失败, 则将该客户端标记为需要移除。
- **BroadcastSystemMessage(const std::string& text) 和 BroadcastChatMessage(const std::string& username, const std::string& text):** 广播一条系统/聊天消息。都是先构造数据帧, 然后调用 `BroadcastFrame` 发送。
- **RegisterClient(const std::shared_ptr<ClientSession>& session):** 注册一个客户端会话。通过加锁保护全局客户端列表; 检查用户名是否已被使用; 如果未被使用, 则将该会话添加到客户端列表中。
- **RemoveClient(const std::shared_ptr<ClientSession>& session):** 移除一个客户端会话。通过加锁保护全局客户端列表; 遍历全局客户端列表, 将指定会话从列表中移除。
- **HandleClient:** 处理一个客户端连接。具体看是调用 `ReceiveLine` 读取 HELLO 帧, 调用 `RegisterClient` 注册会话; 调用 `ReceiveLine` 读取指令帧, 调用 `BroadcastChatMessage` 分发指令; 调用 `ReceiveLine` 读取 BYE 帧, 调用 `RemoveClient` 移除会话并广播离线提示。
- **ConsoleHandler(DWORD signal):** 处理控制台信号 (如 Ctrl+C)。如果接收到 `CTRL_C_EVENT` 或 `CTRL_BREAK_EVENT`, 设置 `g_running` 为 false, 关闭监听套接字。
- **主循环:** `main` 函数中调用设计的函数, 负责初始化和运行主循环, 主要有四个实现:

1. **初始化**: 设置控制台编码为 UTF-8; 解析命令行参数, 配置服务器选项; 初始化 WinSock; 注册控制台信号处理函数。
2. **创建监听套接字**: 创建 TCP 套接字, 设置套接字选项 (地址复用), 并绑定到指定的 IP 和端口, 开始监听连接请求。
3. **主循环**: 循环接受客户端连接, 为每个客户端创建一个 ClientSession 对象, 启动一个线程处理客户端连接 (调用 HandleClient)。
4. **收尾**: 等待所有线程结束, 关闭监听套接字, 清理 WinSock。

3.3 命令行客户端 (client)

ClientOptions 既支持 --host/--port/--name 参数, 也能在互动模式下逐项提示用户输入。主线程在握手通过后启动 ReceiverLoop:

- 后台线程消费服务器帧, 分别对 MSG/SYS/ERR/WELCOME 设置格式化输出, 并在 socket 关闭时把 running 设为 false。
- 前台线程读取终端输入, 支持 “/quit” 发送 BYE, 同时对空消息与超长消息给出提示。
- 所有控制台输出都围绕 g_io_mutex 进行串行化, 确保提示符与消息不会交错。

命令行客户端在握手后会提示 “Connected to ...”, 随后进入如下流程:

1. **参数采集**: 互动模式下 PromptInteractiveOptions 逐项提示 host/port/username, 并循环校验端口范围。
2. **用户名归一**: 调用 chat::NormalizeUsername 去掉非法字符并限制 2-32 字节。
3. **消息发送**: SendClientMessage/SendClientBye 封装 “MSG/BYE <len>” 头部, “/quit” 命令会主动附带 “client quit”。
4. **异常收尾**: ReceiveLine 返回失败即把 running=false, 前台线程检测后退出循环并 shutdown socket。

控制台输出统一为 “ [timestamp] user: payload“ 或 “[SYS]/[ERROR]“ 前缀, 方便和服务端日志对照。为了实现该流程, 设计了多个函数来逐步实现客户端的功能要求:

- **PromptInteractiveOptions(GuiState* state)**: 在交互模式下逐项提示用户输入。包括主机地址、端口号和用户名。并对端口号进行范围校验, 确保在 1-65535 之间。此外还会提示用户输入用户名, 并调用 chat::NormalizeUsername 去掉非法字符并限制 2-32 字节。
- **PrintUsage()**: 打印客户端的使用说明。
- **SendClientMessage(SOCKET socket, const std::string& payload)**: 向服务器发送一条聊天消息。具体是构造消息头部, 格式为 MSG < 消息长度 >, 然后调用 chat::SendLine 发送头部; 调用 chat::SendAll 发送负载。
- **SendClientBye(SOCKET socket, const std::string& reason)**: 发送一个退出消息。具体看是调用 chat::SendLine 发送 “BYE <len>” 头部; 调用 chat::SendAll 发送退出原因。

- **ReceiverLoop(SOCKET socket, std::atomic<bool>& running):** 接收服务器发送的消息，并根据消息类型进行处理。循环接收服务器发送的消息，调用 chat::ReceiveLine 读取消息头部；根据消息类型 (MSG、SYS、ERR、WELCOME) 进行处理，格式化输出到控制台；如果接收到错误或连接关闭，设置 running 为 false，退出循环。

3.4 图形界面客户端 (client_gui)

GUI 版本通过 GuiState 保存窗口句柄、输入框、发送按钮、后台线程句柄以及 socket；若未提供用户名，则借助 LoginDialogState 弹窗收集并校验输入。NetworkThreadProc 负责连接服务器、发送 HELLO、解析所有帧，并把格式化日志用 PostMessageW 投递到 UI 线程的 WM_CHAT_LOG。SendUserInput 则在点击 Send 或回车后读取编辑框、转换为 UTF-8、比较长度并调用 SendPayload。窗口过程 WndProc 动态创建/布局控件，处理 WM_CHAT_* 通知并在 WM_DESTROY 中调用 RequestSocketShutdown，最终等待后台线程结束再释放资源。GUI 版本还细化为以下阶段：

- **登录阶段:** ParseOptions 解析 CLI 参数，缺省时由 LoginWndProc 弹窗并给出 “GuiUserXXXX” 默认值。
- **网络线程:** NetworkThreadProc 在 connect/HELLO 失败时输出 “[ERROR]”，成功则保存 socket 并置 connected=true 通知 UI。
- **日志展示:** 通过 WM_CHAT_LOG+AppendText 滚动输出 “[time] user: payload”，WM_CHAT_CONNECTED 控制 Send 按钮状态。
- **资源回收:** UI 的 WM_DESTROY 调 RequestSocketShutdown，等待后台线程结束后再关闭句柄。

因此 GUI 也能完整呈现 [SYS]/[ERROR] 事件，体验与 CLI 保持一致。

3.5 构建与运行

CMakeLists.txt 为三个可执行程序 and 静态库声明目标，Windows 环境可直接执行：

```
1 cmake -S . -B build -G "Visual Studio 17 2022"
2 cmake --build build --config Release
```

编译完成后，可在 build/Release 中通过以下方式启动：

- 服务器: chat_server.exe --host 127.0.0.1 --port 5555 --name LabServer
- CLI 客户端: chat_client.exe --host 127.0.0.1 --name Alice
- GUI 客户端: chat_client_gui.exe --host 127.0.0.1 --name Bob

4 运行与验证

为验证功能，在 Windows 11 + Visual Studio 2022 的环境中对服务器、CLI 与 GUI 客户端进行了联调，重点考察握手、广播、GUI 交互。

先正常进行编译，然后先后打开一个服务端窗口、两个客户端 cli 窗口及一个客户端 gui 窗口，验证客户端正常接入服务器：

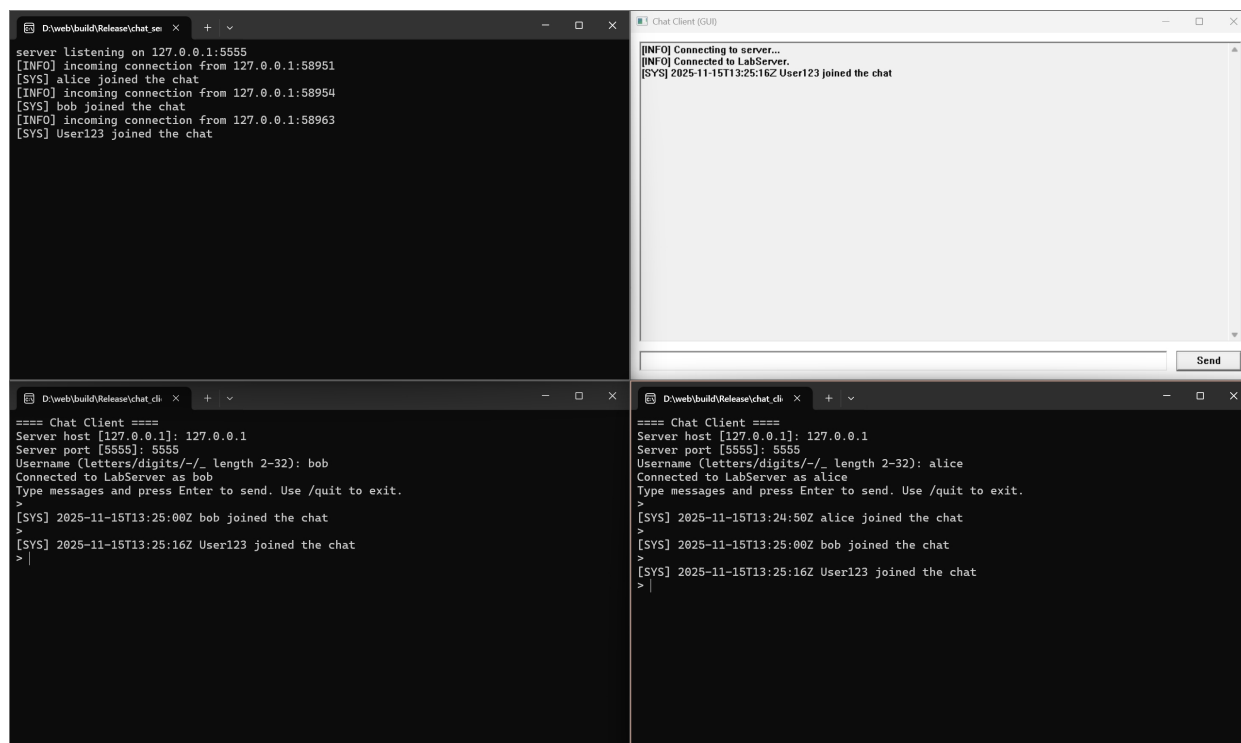


图 4.1: 正常接入

然后分别在一个 cli 客户端和一个 gui 客户端窗口输入消息，验证消息正常发送、多端口能接收并显示：

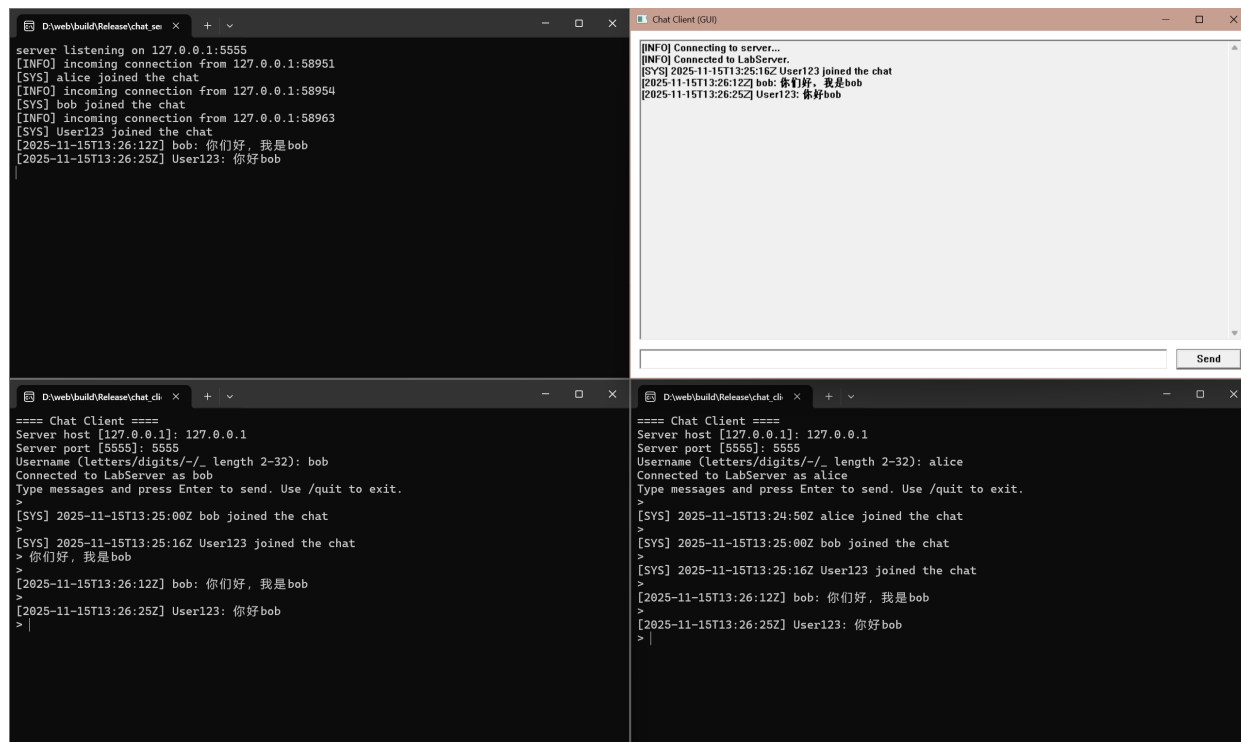


图 4.2: 消息发送与接收

接着分别用”/quit”指令和 Ctrl+C 退出，验证正常退出并释放资源：

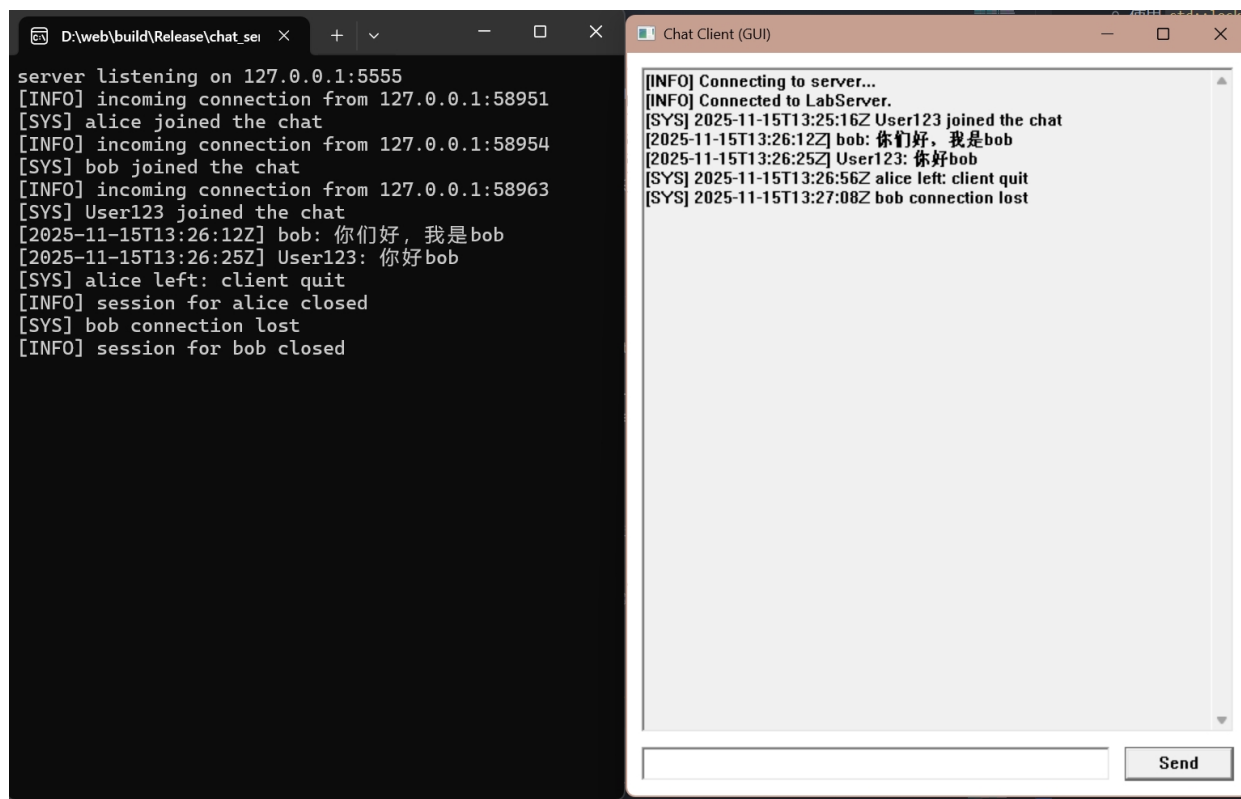


图 4.3: 正常退出

综合上述测试, 说明协议帧解析、UTF-8 支持、多线程同步与收尾终止均按预期工作。

5 实验总结与改进方向

实验过程中从零定义协议、完善多线程同步再到提供 GUI 体验, 使我对 Windows 网络编程、编码规范与跨线程通信有了系统认识, 也体会到良好抽象对 CLI/GUI 共享逻辑的价值。

后续可从以下方面继续改进:

- **安全性:** 引入 TLS 或最少加上消息摘要, 避免明文传输在开放网络中被窃听或篡改。
- **多房间与历史记录:** 在服务器端增加房间概念并持久化最近聊天记录, 客户端上线即可拉取上下文。
- **GUI 体验:** 完善快捷键、通知气泡以及 Emoji 输入等功能, 进一步贴合日常场景。

通过本实验不仅掌握了 Winsock2 API 的常见用法, 也认识到规范化的协议定义、充足的日志与严谨的资源回收是构建稳定网络程序的关键。